



Initial Configuration Manual

Presupuestos AP | Accounts Payable | Fonda Argentina

This manual walks through getting the system ready to operate from scratch: credentials, syncing with Odoo, the expense-type catalog, and - the trickiest part - the mapping that decides which expense type each purchase belongs to. For day-to-day use (dashboard, reports), see the separate **User Guide**.

1. Prerequisites

Before you start, have on hand:

- An Odoo user with read access to vendor bills (`account.move`), payments (`account.payment`), products, and product categories.
- Credentials for the MySQL database the app will run on (dev and/or production).
- Python installed and access to the project folder.

Copy `core/config/.env.example` to `core/config/.env` and fill in these variables:

Variable	What it's for
DJANGO_SECRET_KEY	Django's internal signing key. Generate a new one, never reuse one from another project.
DJANGO_ALLOWED_HOSTS	Domains/IPs the app can be reached from.
PRESUPUESTOS_DB_* (and _DEV)	Host, port, user, password, and name of the MySQL database, for production and for development.
ODOO_URL / ODOO_DB_NAME / ODOO_USER / ODOO_PASSWORD	XML-RPC connection to the same Odoo instance already used by the Wansoft project.

Note: on the development machine, the Django server runs on port 8010 (not the default 8000), because 8000 is already taken by XAMPP.

2. Set up the environment

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
python manage.py migrate
python manage.py createsuperuser
python manage.py runserver 8010
```

You can now log into the admin panel at `http://127.0.0.1:8010/admin/` with the superuser you just created.

3. Sync branches from Odoo

Every branch (sucursal) in the app maps to a company (`res.company`) in Odoo. This script pulls them once (and is safe to re-run later if a new branch is added in Odoo):

```
python scripts/sync_sucursales.py
```

It creates branches that don't exist yet and renames any that changed their name in Odoo. It never deletes a branch, even if it disappears from Odoo - if a branch stops operating, it's deactivated by hand in the admin (`Sucursales`, the **Activa** column).

4. Set up expense categories and expense types

Before mapping anything, the internal expense-type catalog has to exist. This catalog is **independent of Odoo** - it's managed directly in `/admin/`:

1. **Categorías** (the top-level P&L bucket, e.g. "Costo de Ventas" / COGS and "Gasto Operativo" / opex). Fully editable, not hardcoded.
2. **Tipos de gasto**: each belongs to a Categoría. Examples already in use at Fonda Argentina: Alimentos, Bebidas, Empaque y Limpieza, Mantenimiento, Renta y Arrendamiento, Nomina y Personal, Combustibles y Transporte, Servicios Publicos, Servicios Profesionales, Publicidad y Marketing, Administrativo y Oficina, Otros / Sin Clasificar.

Important: the automatic classifier in step 5 looks up Tipos de Gasto **by exact name** (see the list above). If a name doesn't exist yet in the catalog, that row will crash the script. Set up the expense types first, with those exact names, before continuing.

5. The account/category -> expense-type mapping (the tricky part)

Every vendor-bill line in Odoo arrives through one of two different paths, and the system needs to know which one to classify it correctly:

Line type	How the expense type is identified
Purchase tied to a purchase order (goods receipt)	The line's account is the generic clearing account "Goods Received Not Invoiced" - that account name says nothing about the expense type. In this case the system uses the product's category (table <code>Categoria de producto</code> -> <code>Tipo de gasto</code>) to figure out the expense type.
Direct expense (rent, maintenance, payroll, services, etc. - not coming from a purchase order)	The line's own account already identifies the expense type (table <code>Cuenta contable</code> -> <code>Tipo de gasto</code>), no need to look at any product.

In other words: there are **two separate mapping tables** in the admin (`Cuentas contables` -> `tipo de gasto` and `Categorias de producto` -> `tipo de gasto`), and the system decides on its own which one to use based on whether the line carries the goods-received clearing account or not. You don't need to fill in both for the same line; each line only ever uses one.

5.1 Automatic step: run the classifier

This script pulls every product category and every account actually used on paid vendor bills from Odoo, creates the matching row in whichever mapping table it belongs to, and best-guesses the expense type from keywords in the name (e.g. a category named "Carnes" gets auto-assigned to "Alimentos"):

```
python scripts/classify_odoo_catalog.py
```

It's safe to run more than once: it never overwrites an expense type a human already set by hand in the admin, it only fills in rows that are new or still blank. When it finishes, it prints two lists on screen: how many new categories/accounts it found, how many it could guess on its own, and the exact name of the ones it **couldn't classify** (those are left with a blank expense type).

5.2 Manual step: fill in what the classifier couldn't guess

Using the "unclassified" list the script printed, go into the admin and fill in the expense type by hand:

- `/admin/presupuestos/categoriaproductotipogasto/` - filter by an empty `Tipo de gasto`, assign the right expense type to each pending product category.
- `/admin/presupuestos/cuentacontabletipogasto/` - same thing, for pending accounts.

Any expense line whose account or category still has no expense type assigned shows up on the dashboard as **"Sin categoria (sin clasificar)"** - it isn't lost or dropped, it just can't be grouped yet.

When a new account or category shows up in Odoo (a new provider, a new product, a new account), re-run `classify_odoo_catalog.py` from time to time to pick it up, and check the "unclassified" list to fill it in by hand if the name didn't match any known keyword.

6. Enter the monthly budget

The budget is captured **once a month** (not weekly) at `/admin/presupuestos/presupuesto/`, and the system splits it across the days of the month on its own so it can be compared week by week:

- **Sucursal**, **Mes** (any day inside the target month - it gets normalized to the 1st) and **Monto** are required.
- **Tipo de gasto** is optional: leave it blank and that amount gets split evenly across whichever expense types don't already have their own row for that month ("everything else").
- Each sucursal + tipo de gasto + mes combination can only exist once - to correct an already-captured month, edit that same row.

7. Automatic sync (Windows Task Scheduler)

Actual spend (paid vendor bills) is pulled from Odoo with `python scripts/scheduler.py`: it's read-only towards Odoo and only writes to `GastoReal`, using the mapping from step 5 to classify each line. Every run also automatically deletes any row that no longer corresponds to a paid bill in Odoo (for example if a payment was cancelled or reversed), so it doesn't need manual cleanup.

On the development machine this is already scheduled in Windows Task Scheduler, three tasks under the `ControlPresupuestos_AP` - prefix:

Task	When	What it runs
Gastos reales AM	Daily, 5:00 am	<code>scripts/scheduler.py</code>
Gastos reales PM	Daily, 2:00 pm	<code>scripts/scheduler.py</code> again (twice a day)
Catalogos mensual	Day 1 of each month, 4:00 am	<code>scripts/classify_odoo_catalog.py</code> (via <code>scripts/run_classify_odoo_catalog.bat</code>); output goes to <code>logs/classify_odoo_catalog.log</code>

Note: these tasks only run while the user's Windows session is logged in (they don't run as a service) and require XAMPP's MySQL to be up at that time - same as any other use of the app on this development machine.

8. Verify everything is working

1. Go to `/dashboard/`, filter by the branch you just set up and a few recent weeks.
2. Confirm the budget shown matches what was entered in step 6 (prorated by day).
3. Check the "Por tipo de gasto" table: if a lot of spend is landing in "Sin categoria (sin clasificar)", go back to step 5.2 and finish the mapping.
4. Generate the executive PDF report from the "Exportar PDF" button and confirm the numbers add up.

For day-to-day use of the app (reading the dashboard, generating reports, interpreting the comparison chart and the monthly running balance) see the separate **User Guide** document.